

RELEASE CERTIFICATION

HORIZON GRID

Final Release QA Report

Version: 0.1.0

Documentation prepared: 2026-08-17

PREPARED FOR EVALUATION

Table of Contents

HORIZON GRID — Final Release QA Report	3
--	---

HORIZON GRID — Final Release QA Report

1. Executive Summary

This report certifies the "HORIZON GRID" product transformation of the platform (formerly "IOC Intelligence Platform"): a full visible rebrand, two new IOC providers (urlscan.io, Google Safe Browsing), three export-security fixes, a deterministic (non-AI) threat-scoring engine, AI-generation outcome tracking, a new Executive Dashboard and Provider Health page backed by real database aggregation, a reorganized global navigation, a Windows installer fix for a real password-mismatch crash, and a final cross-cutting chaos/performance/red-team pass. Internal identifiers (on-disk data folder name, Postgres database name, Python package names) were deliberately left unchanged to avoid upgrade risk; every user-visible surface was rebranded.

Every workstream below was built, then independently adversarially reviewed by a second pass reading the actual code (not the builder's own summary), then propagated to the installed copy, rebuilt, regression-tested, and live-verified against the real running application. An earlier draft of this report disclosed the scoring-engine anti-flood fix as pending live re-verification because the installed copy's containers were down for a real installer test in progress — that installer test has since completed successfully (the fixed installer correctly resolved the reported password-mismatch failure), the containers are back up on a genuinely fresh install, and the anti-flood fix has now been re-confirmed live on that fresh install (22/22 scoring-engine tests passing, full regression suite 308 passed/0 failed). A second, real gap was found and fixed after that reinstall: the two new providers (urlscan.io, Google Safe Browsing) had no live "Test Connection" wiring at all — confirmed and fixed live, see §9a.

2. Scope Tested

- Backend: FastAPI + SQLAlchemy async ORM + Postgres 16, Redis, Celery worker/beat, Neo4j, OpenSearch.
- Frontend: Next.js 14 App Router, TypeScript, Tailwind, shadcn/radix components.
- Windows installer: Inno Setup 6, PowerShell wizard/scripts.
- Every file touched by this mission (rebrand strings, 2 new provider modules, `app/scoring/engine.py`, `app/ai/schemas.py` / `service.py` outcome tracking, `app/core/dashboard.py`, `app/api/routes/dashboard.py`, `app/ai/dashboard_summary.py`, the new frontend dashboard/provider-health pages and components, `WorkspaceNav.tsx`, `app/core/db.py` / `config.py` connection pooling, `windows/scripts/Common.ps1` / `wizard/Setup-Wizard.ps1`).

3. Feature Matrix (requirement → verification → result)

Requirement	Verification	Result
Full visible rebrand to HORIZON GRID	Grep sweep across backend/frontend/installer/docs + dedicated red-team rebrand sweep of every new surface added later in the mission	PASS
Internal identifiers left unchanged (upgrade safety)	Confirmed DB name, ProgramData folder name, k8s namespace, package names untouched	PASS
urlscan.io + Google Safe Browsing providers	27 unit tests; live investigation with real HTTP calls; live chaos test with deliberately invalid credentials	PASS
A provider failure never reads as "safe"	Live test: invalid API keys → <code>status=error</code> , Safe Browsing <code>verdict=unknown</code> , <code>final_verdict=UNKNOWN</code> (never benign/clean)	PASS
CSV/PDF export security fixes (BUG-022/023/024)	Reproduced original bugs, confirmed fixed, regression-tested	PASS
Deterministic scoring computed before any AI call	Code trace + live investigation showing real breakdown	PASS
AI can narrate but never override the score	<code>service.py</code> re-validates the FULL <code>FinalAssessment</code> against the spliced-in deterministic <code>RiskAssessment</code> instance (closes a real validator-ordering gap found during build)	PASS
Scoring audit trail (version + factor breakdown persisted)	Found missing during build (engine computed it, <code>service.py</code> discarded it before persisting) — fixed, live-verified	PASS (fixed)
AI-outcome tracking (success vs. correct skip vs. genuine failure)	New <code>ai_outcome</code> column + migration, backfilled against 72 live rows, verified against real historical data before choosing the backfill rule	PASS
KPI dashboard — every number real, never hardcoded	Live browser screenshots against real data; adversarial correctness review	PASS
Provider Health — never reports "healthy" with zero evidence	Dedicated test + live trace; a provider correctly reporting "nothing found" must count as healthy, not a failure — real bug found (NO_DATA/UNSUPPORTED_IOC miscounted as failures) and fixed, confirmed live against a real provider (OTX moved from reported "degraded"/64% to correctly "healthy"/100%)	PASS (fixed)
Dashboard/Provider Health RBAC (new <code>dashboard:read</code> , all 3 roles)	Adversarial privilege-escalation review + independent RBAC cross-cutting sweep, both clean	PASS
Existing <code>getProviderHealth()</code> caller not broken by the richer response shape	Real regression found (dropped <code>supported_types</code> , would have crashed the live investigation-launch page) — fixed same day, regression test added	PASS (fixed)
Executive Dashboard + Provider Health frontend	Real browser screenshots against live data; null-vs-zero rendering traced in code	PASS
AI executive summary never fabricates a number	Code review of prompt discipline + forced-failure test proving the template fallback is real and number-accurate	PASS
Global nav reorganized into named groups, no dead links	Every route cross-checked against real <code>page.tsx</code> files; active-route highlighting traced for every real route including a prefix-collision fix (<code>/dashboard</code> vs <code>/dashboard/provider-health</code>)	PASS
No neon/glow/sci-fi excess in the UI	Grepped for glow/shadow/neon classes — zero hits; confirmed the existing "Enterprise SOC dark theme" tokens were reused, not replaced	PASS
KPI/Provider-Health endpoints survive concurrent dashboard loads	Real failure found: 25 concurrent requests to Provider Health = 100% timeout. Root-caused to (a) ~306 sequential DB round-trips per request, (b) connection-pool sizing not accounting for 2	PASS (fixed)

Requirement	Verification	Result
	connections held per authenticated request. Both fixed; confirmed live: same load now completes in 1.6s	
Scoring engine resistant to manipulation via crafted provider data	Real vulnerability found and fixed: a single free, unprivileged OTX/ThreatFox/MalwareBazaar community account could flood the correlation component with distinct fabricated edges and push any indicator's score into "high" severity with zero real infrastructure or corroboration. Fixed by applying the same anti-flood corroboration discount already used for provider votes; unit-tested (22/22 passing); live-verified on the fresh post-reinstall container	PASS (fixed)
Installer: fresh install never crashes on a Postgres password mismatch	Root-caused a real reported failure (backend crash-loop on Postgres auth error) to a fresh install generating a new random password while an old, incompatible database volume survives from an abandoned attempt. Fixed by detecting and clearing an orphaned volume before a genuinely fresh install; installer recompiled. The user then ran the real installer and confirmed it succeeded — the fresh install came up healthy with no password error	PASS (fixed, user-confirmed)
urlscan.io / Google Safe Browsing "Test Connection" button has a real, working check	Real gap found post-reinstall: both new providers returned "has no live connection test" on every Test Connection click — the dispatcher (<code>app/providers/connection_test.py</code>) never had entries for them, even though the real investigation-time provider logic was already correct and tested. Fixed by adding real checks mirroring each provider's own live <code>fetch()</code> call; confirmed live against the real running container	PASS (fixed)

4. RBAC Results

Full cross-cutting sweep (not just per-endpoint spot checks) of every mission-touched route against the current `ROLE_PERMISSIONS` matrix, independently re-verified by a second adversarial pass. No permission string mismatch, no docstring-vs-code discrepancy, no role silently missing a `ROLE_PERMISSIONS` entry, no wider/narrower access than intended anywhere in the chain from route → dependency → permission string → role table. **Clean.**

5. Provider Results

18 registered providers, including the 2 new this mission. Real live data confirms the health-reporting fix: providers with genuine errors (VirusTotal, AbuseIPDB — 26 consecutive real errors from an earlier, unrelated credential issue) correctly report "down"; providers legitimately returning "nothing found" for most lookups (OTX, WHOIS/RDAP, Spamhaus, Internet Intelligence Collector) correctly report "healthy" at 100%. Both new providers confirmed to never report a failure as "safe," live, with real invalid-credential HTTP calls against the real external APIs.

6. Scoring Engine Results

Deterministic, computed before any AI call, versioned (`SCORING_ENGINE_VERSION`), audit-trailed (version + full factor breakdown now persisted on every assessment). One real manipulation vulnerability found via adversarial red-team (numerically reproduced by two independent reviewers) and fixed same day: correlation-graph evidence from a single, unprivileged source could previously saturate the correlation component with no cross-provider corroboration requirement, unlike the provider-vote path which already had this defense. Fixed by applying the identical corroboration-discount philosophy to correlation edges, keyed on distinct asserting providers. 22/22 unit tests passing including the new regression test proving the fix discriminates a real flood from genuine multi-provider corroboration (which is deliberately NOT penalized).

One disclosed, non-blocking hardening item: `_provider_votes()` would treat a NaN/Infinity value as full-strength "malicious" rather than rejecting it, but no live provider today produces such a value (VirusTotal, the only provider populating the affected fields, derives them from its own server-computed stats, not attacker-controllable text). Recommended follow-up, not release-blocking.

7. Dashboard / Provider Health Results

Every KPI and every provider-health field traced to a real database query with no hardcoded value. Two real bugs found and fixed during this mission (NO_DATA-as-failure miscounting; the `supported_types` backward-compatibility regression) — both confirmed fixed against real live data. Null-vs-zero distinction (a rate/latency that's genuinely absent renders "N/A", never a fabricated 0%) traced in both backend query logic and frontend render logic.

8. Performance / Concurrency Results

Real load testing (not simulated) against the live rebuilt container found and fixed a genuine complete-failure mode: 25 concurrent requests to `GET /providers/health` timed out 100% of the time. Root causes and fixes:

1. `get_provider_health_history()` issued ~306 sequential DB round-trips per request (4 metrics × 4 windows × 18 providers) — consolidated into ~19 total via SQL conditional aggregation, one query per provider window-set instead of sixteen.
2. The DB connection pool was sized for SQLAlchemy's un-tuned defaults (5+10) and didn't account for every authenticated request holding 2 connections simultaneously (one for the auth dependency, one opened separately by the service function) — resized per-process-role (`app-backend` gets 30+20=50, since it's the only process serving concurrent user traffic; `celery_worker` / `celery_beat` keep a conservative 5+5=10 each, since they barely touch the database) — sized so the worst-case combined total (70) stays safely under Postgres's 100 `max_connections` .

Confirmed live after both fixes: the same 25-concurrent-request test that previously failed 100% of the time now completes in 1.6 seconds, 100% success.

One disclosed, non-blocking, environmental limitation: the AI-generated executive-summary endpoint can become slow under heavy *concurrent* load specifically when a local Ollama model is configured, since a single local model serializes generation requests — this is an infrastructure/model-concurrency characteristic, not an application bug, does not affect data correctness, and does not affect the Dashboard's core KPI tiles or the Provider Health page (both pure database reads).

9. Installer Results

A real, user-reported failure (backend crash-loop on a Postgres authentication error immediately after install) was root-caused to a genuine architectural gap: a fresh install with no existing `.env` always generates a brand-new random database password, but Postgres only ever applies that password to an empty, uninitialized data directory — if a database volume from an earlier, abandoned install attempt survives on the machine, the new password can never match it. Fixed by detecting and removing an orphaned database volume (via Docker Compose's own project/volume labels, not a hardcoded name) specifically and only on a genuinely fresh install — an upgrade/reconfigure of a real existing install is untouched and continues to correctly reuse its real existing password. The installer was recompiled from the fixed source.

User-confirmed real-world result: the user then ran the rebuilt installer themselves (this process does not perform destructive install actions directly, per established practice). The first retry hit an unrelated Docker Desktop build-cache corruption error (`failed to prepare extraction snapshot ... parent snapshot ... does not exist`) — a Docker Desktop/containerd infrastructure issue, not a HORIZON GRID defect, confirmed by the error occurring identically across all three Python-based service images from a shared corrupted base-layer cache entry. Resolved by a full Docker Desktop + WSL2 restart (with explicit user approval before either the Docker Desktop restart, which affected unrelated containers, and the broader `ws1 --shutdown`). The subsequent install attempt succeeded cleanly: all 8 containers came up healthy, and the fresh install showed no password error.

9a. Post-Reinstall Findings

Two real issues were found and fixed on the fresh, user-installed instance, after the QA cycle above had already completed against the previous instance:

1. **urlscan.io / Google Safe Browsing "Test Connection" had no live check.** Both new providers' Test Connection button unconditionally returned `"'<provider_id>' has no live connection test"` — the shared IOC-provider connection-test dispatcher (`app/providers/connection_test.py`) was never given entries for either provider when they were added, even though their real investigation-time provider logic (`app/providers/urlscan_io.py` , `app/providers/google_safe_browsing.py`) was already correct and covered by 27 unit tests. This did not affect real investigations (which use the already-correct `fetch()` path), only the standalone credential-testing button. Fixed by adding `_check_google_safe_browsing` / `_check_urlscan` functions mirroring each provider's own real API call exactly, and wiring both into the dispatcher. Confirmed live: both now return a real result from a real HTTP call to the real external API instead of the generic "no test" message.
2. **A related, separate finding, not a bug:** the user's real Google Safe Browsing key returned a genuine HTTP 403 once the connection test actually ran, with Google's own error message identifying the cause precisely ("Method doesn't allow unregistered callers... Please use API Key or other form of API consumer identity to call this API") — the standard Google Cloud message for the Safe Browsing API not yet being enabled on the key's project, a one-time setting in Google Cloud Console separate from generating the key itself. Not a platform defect; documented here because it was a real, live-diagnosed result during this QA cycle.

Also worth noting, not a defect: the "Test Connection" button's behavior of requiring an already-saved credential to be retyped before it can be tested (rather than falling back to the stored value) is a **deliberate security property**, confirmed by reading `app/api/routes/providers.py`'s own docstring ("never persisted, never read from settings") — credentials are never round-tripped back out to the browser once saved. This surprised a real user during this QA cycle and is now called out explicitly in the Admin Guide and Troubleshooting documentation so it doesn't surprise the next one.

10. Security Results (of the platform itself)

- CSV formula-injection and PDF markup-injection export bugs (BUG-022/023) fixed and regression-tested.
- Export permission-gate bug (BUG-024, exported on `lookup:read` instead of `lookup:export`) fixed.
- New `dashboard:read` permission is deliberately broad (read-only, all 3 roles) by design, confirmed to grant no access to any credential, per-user, or otherwise-gated field.
- Scoring-engine manipulation vulnerability (§6) found and fixed.
- No SQL injection risk in any new query (all parameterized via SQLAlchemy Core, confirmed in the query-optimization review).
- No new npm dependency was introduced anywhere in the frontend work this mission.

11. Documentation Results

Documentation source files updated to describe every new user-facing feature (Executive Dashboard, Provider Health page, the 2 new providers, real load-tested performance characteristics, the disclosed AI-concurrency limitation) in the existing evidence-based narrative style. Fresh screenshots of the new UI are pending the installed copy's containers coming back up (see §22) — the doc source files use `[FIGURE: ...]` placeholders ready to be filled in via the existing screenshot pipeline once that's possible.

12. Test Statistics (most recent full runs)

- Dev-tree host venv, full suite, before the final scoring-engine anti-flood fix: **337 passed, 3 failed, 5 errors, 1 deselected** (all failures/errors are the same pre-existing, documented, unrelated issues carried across this entire mission — `test_lookup_flow.py`'s concurrency/cache tests and `test_lookup_stream_persistence.py`'s async-teardown issue — never caused by any change made this mission).
- Dev-tree, scoring-engine unit tests only, WITH the anti-flood fix: **22 passed, 0 failed** (up from 21 — the new regression test).

- Installed-copy container, full suite, after the connection-pool/query-optimization fixes (before the anti-flood fix, on the pre-reinstall instance): **307 passed, 0 failed, 38 skipped** (skips are host-only tests requiring services not reachable from inside the container's network namespace — a known, pre-existing environment characteristic, not a failure).
- **Installed-copy container, full suite, on the fresh post-reinstall instance, WITH the anti-flood fix AND the connection-test wiring fix: 308 passed, 0 failed, 38 skipped.** Scoring-engine tests specifically re-run in this same fresh container: 22 passed, 0 failed.
- Live load test, `GET /providers/health`, 25 concurrent requests: before the fix, 100% timeout at 30s; after the fix, 100% success in 1.6s (re-confirmed on the fresh post-reinstall instance).

13. Known Limitations (disclosed, not release blockers)

1. The AI-generated executive summary can be slow under heavy concurrent load with a local Ollama backend configured (§8) — infrastructure characteristic, not a bug; does not affect data correctness or the dashboard's core KPI/provider-health surfaces.
2. `_provider_votes()`'s NaN/Infinity handling is a latent hardening gap with no live exploitable path today (§6) — recommended follow-up.
3. The "Test Connection" button never falls back to an already-saved credential (§9a) — a deliberate security property, not a bug, but confirmed to be a real point of user confusion; now documented explicitly.

14. Resolved Since Initial Draft

An earlier draft of this report disclosed the anti-flood fix's live re-verification and the installer's real-world test as pending, blocked on a concurrent installer test the user was running. Both have since completed:

- The anti-flood fix is now live-verified on the fresh, user-installed instance (§12).
- The user ran the real installer, hit and worked through an unrelated Docker Desktop infrastructure issue (§9), and confirmed the install succeeded with no password error.
- One further real issue (missing Test Connection wiring for the 2 new providers, §9a) was found post-reinstall and fixed and live-verified in the same session.

Fresh documentation screenshots against the current UI remain the one item still in progress as of this report.

15. Final Risk Assessment and Verdict

Applying this mission's own non-negotiable release-blocker rules (any unresolved P0/P1, confirmed auth bypass, privilege escalation, cross-user data leak, credential leak, command injection, DB corruption; and this mission's own additional rules: a dashboard showing incorrect real-world metrics, a provider incorrectly reporting HEALTHY, a Safe Browsing failure read as SAFE, or an AI override of the deterministic score):

- No auth bypass, privilege escalation, cross-user leak, credential leak, command injection, or DB corruption was found anywhere in this mission's surface, confirmed by two independent cross-cutting reviews.
- Every dashboard/KPI number was confirmed live to be real, never hardcoded, and every "incorrect metric" bug found during the mission itself (NO_DATA-as-failure, `supported_types` regression) was fixed and confirmed live.
- A provider is confirmed, live, to never report HEALTHY with zero evidence, and a Safe Browsing/urlscan failure is confirmed, live, to never read as SAFE.
- The AI is confirmed, by code trace and a real forced-failure test, to never override the deterministic score.
- The one genuine scoring-manipulation vulnerability found (§6) has been fixed, unit-tested, and — as of this update — live-verified end-to-end on a fresh, real, user-installed instance.

- The installer itself has now been run for real by the user, on a genuinely fresh machine state, and confirmed to work.

VERDICT: RELEASE READY WITH KNOWN LIMITATIONS

Every substantive finding in this report has been fixed, tested, and live-verified against the real running application, including a real end-to-end installer run performed by the user. The remaining known limitations (§13) are disclosed, non-blocking, and do not affect correctness, security, or the platform's core guarantees.